

Computer Organization, Spring 2026

Project Assignment 2 : Single Cycle CPU Design

Lecturer: 陳雅淑教授

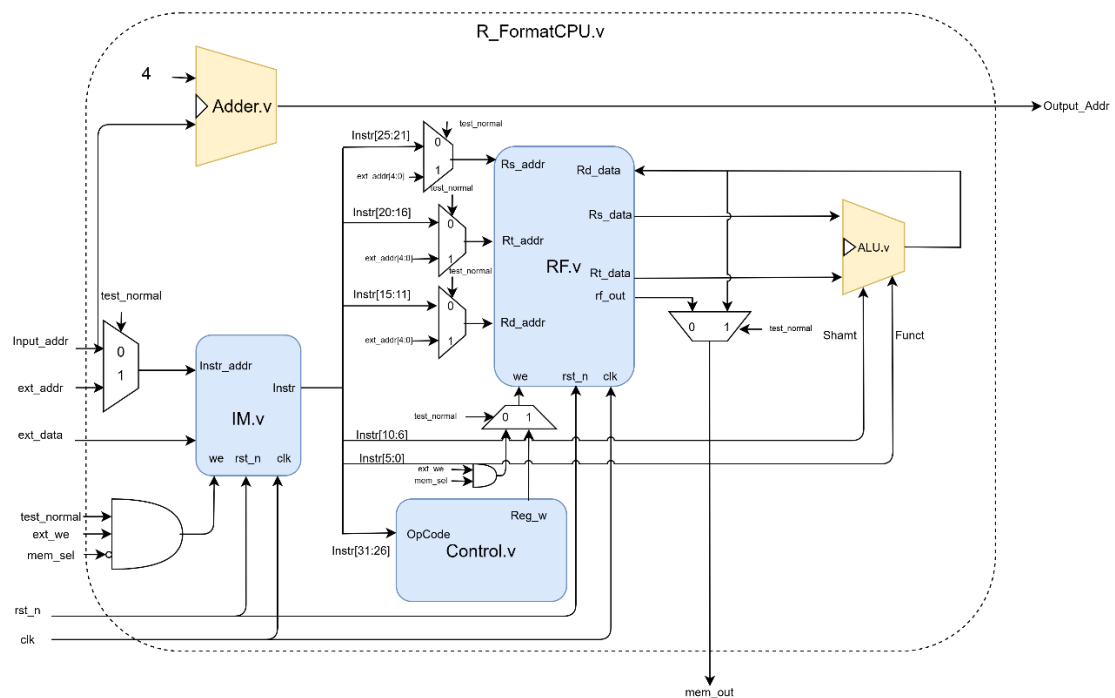
Department: Electrical Engineering

Student ID: B11230024

Name: 王映媛

1.Design Process

Part1.R_FormatCPU



(a)Adder.v

Adder 是一個純組合邏輯模組，負責計算下一個程式計數器的值（PC + 4）

(b)IM.v

IM 是一個 128 Bytes 的 Big-endian 記憶體，內部以 128 個 8-bit 暫存器陣列（mem[0:127]）儲存指令。讀取時將連續 4 個 Byte 拼接為 32-bit 指令字組。

寫入邏輯：

rst_n = 0：所有 mem 清零

test_normal=1 且 ext_we=1 且 mem_sel=0：Testbench 將 ext_data 以 Big-endian 格式寫入 mem[ext_addr] 至 mem[ext_addr+3]

讀出邏輯：

test_normal = 0：輸出 {mem[Input_Addr], mem[Input_Addr+1], mem[Input_Addr+2], mem[Input_Addr+3]}

test_normal = 1（測試模式）：輸出以 ext_addr 為基底的 4 Bytes，供 TB 讀取

驗證

(c)RF.v

RF 包含 32 個 32-bit 暫存器，支援非同步讀取 Rs、Rt 與同步寫入 Rd。測試模式外部直接寫入，rf_out 輸出供 R-FormatCPU 的 TB 讀取暫存器內容。

寫入：

rst_n = 0：所有暫存器清零

test_normal=1, ext_we=1, mem_sel=1：寫入 regs[ext_addr] (R-format 測試用)

test_normal=0, Reg_w=1：正常執行模式，寫入 regs[Rd_addr](硬體保護\$0，Rd_addr ≠ 0 才寫入)

讀出：

Rs_data = regs[Rs_addr]，Rt_data = regs[Rt_addr]，rf_out = regs[ext_addr]。

(d)ALU.v

純組合邏輯電路，根據 Funct_ctrl 編碼執行特定的運算 addu, subu, OR, SLL。由於 R-format CPU 不需要 I-format 的 ALU_op 多工邏輯，因此我直接拿掉 ALU_Control 模組，直接將 Instr[5:0]的 MIPS 原始 Funct 送進 ALU，內部的 case 直接對應 MIPS 標準 Funct 編碼 (100001=Addu、100011=Subu、000000=Sll、100101=OR)

(e)Control.v

解碼指令的 OpCode 欄位 (Instr[31:26])。在 R-format 指令 (OpCode 為 0) 時，將寫回致能訊號 Reg_w 設為有效。

(f)R_FormatCPU.v 將以上模組以架構圖所示連接

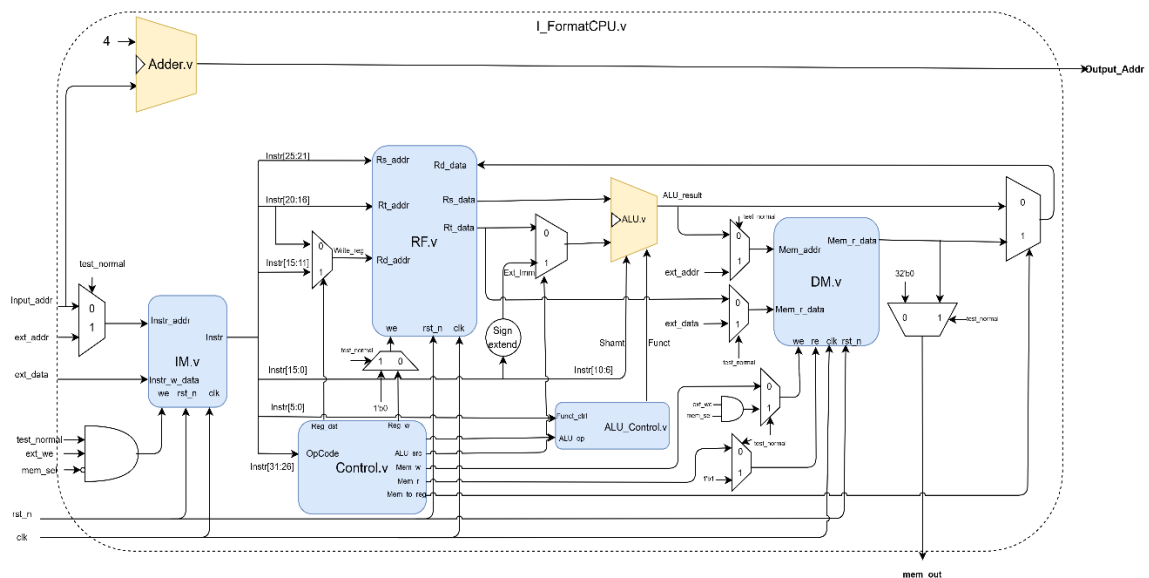
1. Control Path

當指令從 IM 輸出，OpCode 進入 Control 模組。若辨識為 R-type 指令，控制單元會發出 Reg_w 訊號，授權 RF 在時脈正緣時將 ALU 的運算結果存入目標暫存器。同時，test_normal 訊號作為全域切換開關，控制多工器選擇是要執行當前指令，還是接受外部測試資料的讀寫。

2. Datapath

1. IF: 根據 Input_Addr 從 IM 取出 32 位元指令，同時 Adder 計算 PC + 4，輸出至 Output_Addr
2. ID: 指令拆解為 Rs、Rt、Rd 位址。透過多工器選擇的位址進入 RF，輸出對應暫存器的 Rs_data 與 Rt_data。
3. EX: ALU 接收兩組暫存器資料，並根據指令的 Funct 欄位與 Shamt (位移量) 欄位進行運算。
4. Write Back): ALU 運算結果 Rd_data 流回 RF 的資料輸入，若 Reg_w 有效，則在下一個時脈週期完成存儲。

Part2.I_FormatCPU



(a)Adder.v(b)IM.v 與 R_FormatCPU 完全相同

(c)RF.v

Rd_addr 由 MUX 決定：Reg_dst=1 選 Instr[15:11] (Rd, R-format)，Reg_dst=0 選 Instr[20:16] (Rt, I-format)

(d)ALU.v

ALU 左側有一個 32-bit MUX，由 ALU_src 控制選擇 Rt_data 或立即數 (Sign/Zero Extend 後的值) 作為第二運算元。Shamt 直接來自 Instr[10:6]

純組合邏輯，根據 ALU_Control 傳入的 Funct 內部編碼執行運算：

Funcnt (內部編碼)	操作	對應指令
001001	Src_1 + Src_2	Addu, addiu, lw, sw
001010	Src_1 - Src_2	Subu
100001	Src_2 << Shamt	Sll
100101	Src_1 Src_2	OR, ori

(e) ALU_Control.v

將 ALU_op + Funcnt_ctrl (Instr[5:0]) 轉換為 ALU 的內部 Funcnt 編碼：

ALU_op	行為
2'b10 (R-format)	查 Funcnt_ctrl 轉為內部編碼
2'b00 / 01 (I-format 加法)	輸出 001001
2'b11 (ori)	輸出 100101

(f) Control.v

依 Instr[31:26] (OpCode) 產生所有控制訊號：

指令	OpCode	Reg_dst	Reg_w	ALU_src	ALU_op	Mem_w	Mem_r	Mem_to_reg
R-format	000000	1	1	0	10	0	0	0
addiu	001001	0	1	1	00	0	0	0
lw	100011	0	1	1	00	0	1	1
sw	101011	—	0	1	00	1	0	—
ori	001101	0	1	1	11	0	0	0

(g) DM.v(結構類似 IM.v)

寫入：

test_normal=1, ext_we=1, mem_sel=1：TB 載入初始 DM 資料

test_normal=0, Mem_w=1：CPU 執行 sw，以 ALU_result 為位址、Rt_data 為寫入資料

讀出：

test_normal=1：以 ext_addr[6:0]讀出，TB 透過 mem_out 驗證結果

test_normal=0：以 Mem_addr 讀出，供 lw 指令使用

(h) I_FormatCPU.v 將以上模組以架構圖所示連接

1. Control Path

控制單元根據 OpCode 產生以下信號來導引資料：

- Reg_dst：

決定寫入目標是指令的[20:16] (Rt)還是[15:11] (Rd)。ALU_src：選擇 ALU 的第二個運算元是來自暫存器的 Rt_data 還是擴充後的立即值 Ext_Imm

- Mem_to_reg：決定寫回暫存器的資料是來自 ALU 運算結果還是記憶體讀取內容
- Reg_w：在 sw 指令時應為 0，其餘需要寫回結果的指令則為 1
- Mem_w：控制是否將資料寫入資料記憶體 (DM)。僅 sw 指令需要寫入記憶體，因此 Mem_w=1；其餘所有指令 Mem_w=0，確保記憶體內容不被誤改。

2. Datapath

1. IF: 根據 Input_Addr 從 IM 取出 32 位元指令，同時 Adder 計算 PC + 4，輸出至 Output_Addr

2.ID:

Instr[31:26] → Control，產生所有控制訊號

Instr[25:21]、Instr[20:16] → RF，讀出 Rs_data、Rt_data

Write_Reg MUX：Reg_dst=1 選 Instr[15:11] (R-format)，Reg_dst=0 選 Instr[20:16] (I-format)

3.EX:

Sign/Zero Extend：ori (OpCode=001101) 使用零擴展，其餘 I-format 使用符號擴展

ALU_src MUX：ALU_src=0 選 Rt_data，ALU_src=1 選 Ext_Imm

ALU_Control 轉換 Funct，ALU 執行運算輸出 ALU_result

4.MEM：

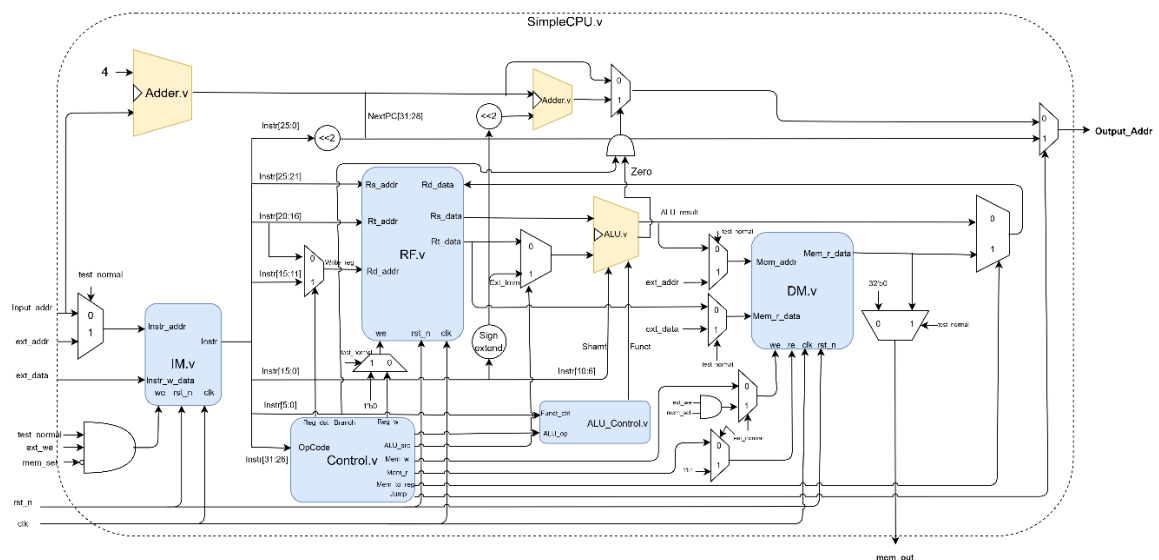
lw：ALU_result 為位址，DM 讀出 Mem_r_data

sw：Mem_w=1，將 Rt_data 寫入 DM[ALU_result]，其他指令不存取 DM

5.WB：

Mem_to_reg=1 (lw) 選 DM 讀出值，否則選 ALU 結果，寫入 RF

Part3.SimpleCPU



SimpleCPU 是在 I_FormatCPU 基礎上新增 BEQ (Branch if Equal) 與 Jump 指令。主要差異在於 PC 的計算路徑從單純的 PC+4 擴展為三路選擇 (PC+4、Branch 目標、Jump 目標)，以及 Control 新增兩個控制訊號 (Branch、Jump) 和 ALU 新增 Zero 旗標輸出

(a.)RF.v(b) DM.v (c)Adder.v(d)IM.v 與 I_FormatCPU 完全相同，邏輯無變更

(e) ALU.v

純組合邏輯，相較 I_FormatCPU 新增 Zero 旗標輸出供 BEQ 使用

Funct	操作	對應指令
001001	Src_1 + Src_2	Addu、addiu、lw、sw
001010	Src_1 - Src_2	Subu、BEQ (比較是否相等)

100001	Src_2 << Shamt	Sll
100101	Src_1 Src_2	OR、ori

(f) ALU_Control.v

將 ALU_op 轉換為 ALU 的內部 Funct 編碼，BEQ 使用 ALU_op=01，ALU 執行 Rs-Rt，若結果為零則 Zero=1，表示兩數相等。

(g) Control.v

相較 I_FormatCPU 新增 Branch 與 Jump 兩個輸出訊號

指令	OpCode	Reg_dst	Reg_w	ALU_src	ALU_op	Mem_w	Mem_to_reg	Branch	Jump
R-format	000000	1	1	0	10	0	0	0	0
addiu	001001	0	1	1	00	0	0	0	0
lw	100011	0	1	1	00	0	1	0	0
sw	101011	0	0	1	00	1	0	0	0
ori	001101	0	1	1	11	0	0	0	0
beq	000100	0	0	0	01	0	0	1	0
jump	000010	0	0	0	00	0	0	0	1

(h) SimpleCPU.v 將以上模組以架構圖所示連接

1. Control Path

控制單元根據 OpCode 產生以下信號來導引資料：

- Reg_dst：

決定寫入目標是指令的[20:16] (Rt)還是[15:11] (Rd)。BEQ、Jump、sw 不寫暫存器，所以這個訊號對它們沒意義（Reg_w=0 時 RF 不寫入）

- ALU_src：選擇 ALU 的第二個運算元是來自暫存器的 Rt_data 還是擴充後的立即值 Ext_Imm
- Mem_to_reg：決定寫回暫存器的資料是來自 ALU 運算結果還是記憶體讀取內容

- Reg_w：在 sw 指令時應為 0，其餘需要寫回結果的指令則為 1
- Mem_w：控制是否將資料寫入資料記憶體 (DM)。僅 sw 指令需要寫入記憶體，因此 Mem_w=1；其餘所有指令 Mem_w=0，確保記憶體內容不被誤改。
- Branch：標示當前指令是否為條件分支指令。僅 BEQ 時 Branch=1，但 Branch 本身不足以決定是否跳轉，必須與 ALU 的 Zero 旗標做 AND。若 $R_s - R_t = 0$ 則 Zero=1，PCSrc=Branch & Zero=1，PC 才會跳至 Branch 目標位址；若兩數不相等則 Zero=0，PC 仍走 PC+4。
- Jump：標示當前指令是否為無條件跳轉指令。僅 Jump 時 Jump=1，此時不論 Branch 與 Zero 的結果為何，Output_Addr 直接選擇 Jump 目標位址 {PC+4[31:28], Instr[25:0], 2'b00}，Jump 的優先權高於 Branch。

2. Datapath

1. IF: 取指令 Input_Addr 送入 IM，非同步讀出 Instr，同時 Adder1 計算 PC+4。

2.ID:

- Instr[31:26] → Control，產生所有控制訊號 (含 Branch、Jump)
- RF 讀出 Rs_data、Rt_data
- Write_Reg MUX：Reg_dst=1 選 Instr[15:11] (R-format)，=0 選 Instr[20:16] (I-format)
- Sign/Zero Extend：ori 使用零擴展，其餘 I-format 使用符號擴展

3. EX:

- ALU_src MUX：ALU_src=0 選 Rt_data，=1 選 Ext_Imm
- BEQ：ALU_src=0，ALU 執行 $R_s - R_t$ ，輸出 Zero 旗標
- ALU_Control 轉換 Funct，ALU 計算輸出 ALU_result
- 同時 Adder2 計算 Branch 目標：PC+4 + (Imm16 左移 2 位符號擴展)

4. MEM:

- lw：DM 以 ALU_result 為位址非同步讀出

- sw : Mem_w=1 , posedge clk 寫入
- BEQ / Jump : 不存取 DM

5. WB:

assign Rd_data = Mem_to_reg ? Mem_r_data : ALU_result;

BEQ 和 Jump 的 Reg_w=0 , 不寫回 RF 。

2. Behavioral Simulation

BEQ (Branch if Equal) 作為最代表性的指令，它同時驗證：

1. ALU 的減法與 Zero 旗標
2. Control Path 的 Branch 訊號
3. Datapath 的 PC 選擇邏輯 (三路 MUX)

(a)IM.dat

```

24 // [0x00] addiu $1, $0, 5 → $1=5
01
00
05
24 // [0x04] addiu $2, $0, 5 → $2=5
02
00
05
10 // [0x08] beq $1,$2,+2 → 5==5 TAKEN → 跳到 0x14
22
00
02
24 // [0x0C] addiu $3,$0,0xFF → FAIL if executed
03
00
FF
AC // [0x10] sw $3, 0($0) → FAIL if DM[0]!=0
03
00
00
24 // [0x14] addiu $4,$0,0x77 → BEQ taken 落點
04
00
77
AC // [0x18] sw $4, 4($0) → DM[4]=0x77 ✓
04
00
04
00 // [0x1C] nop
00
00
00

```

(b) Vivado tcl console

```

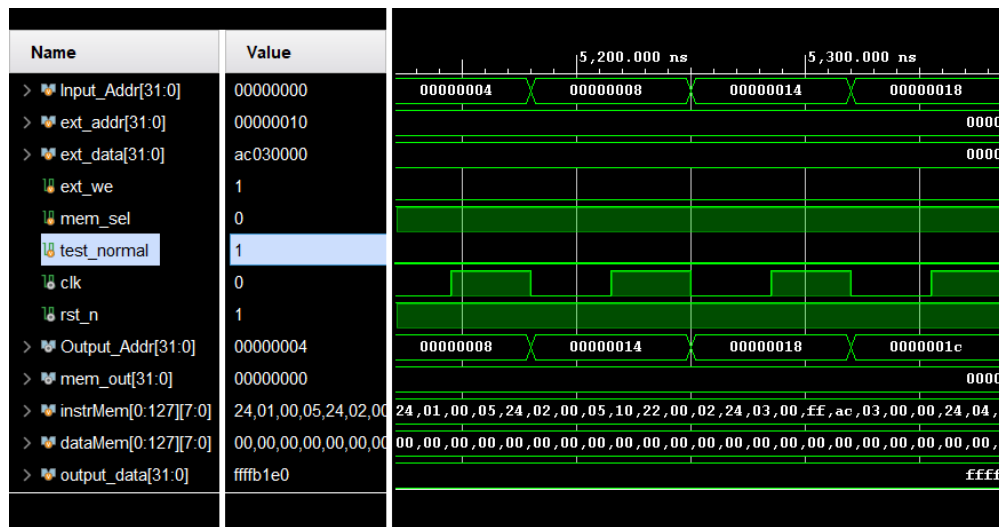
--- [Data Memory Results] ---
DM[ 0] : 00000000
DM[ 4] : 00000077
DM[ 8] : 00000000
DM[12] : 00000000

```

DM[0] = 00000000 : 確認 FAIL 指令從未執行，BEQ 確實跳轉

DM[4] = 00000077：確認跳轉目標位址 0x14 計算正確，落點指令正常執行

(c) hexadecimal waveforms



由波型圖可見 Input_Addr=00000008(當前 PC，指向 BEQ 指令)，
Output_Addr= 00000014(跳轉成立，目標為 0x14 (非循序的 0x0C))

指令：beq \$1, \$2, +2；機器碼：0x10220002

解碼：

OpCode = 000100 =>Control 設定 Branch=1, ALU_op=01, ALU_src=0

Rs= \$1 (值為 5)、Rt= \$2 (值為 5)、Imm= 2

Control Path：

Branch = 1

ALU_op = 01 =>ALU_Control 輸出 Sub (001010)

Datapath：

Src_1 = regs[1] = 5、Src_2 = regs[2] = 5 (ALU_src=0，選 Rt_data)

ALU = 5 - 5 = 0、Zero = 1

PCSrc = Branch & Zero = 1 & 1 = 1

PC 計算：

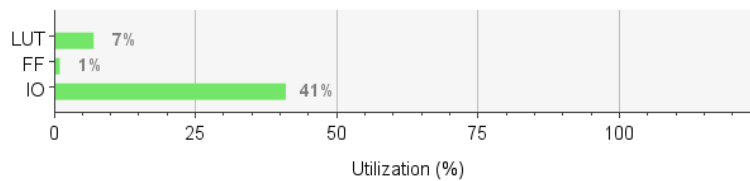
PC+4 = 0x08 + 4 = 0x0C

$\text{Branch_Addr} = 0x0C + (2 \ll 2) = 0x0C + 8 = 0x14$

$\text{Output_Addr} = \text{Branch_Addr} = 0x14$

3. Synthesis Result (part 3)

Resource	Utilization	Available	Utilization %
LUT	9045	134600	6.72
FF	3072	269200	1.14
IO	165	400	41.25



Hardware Utilization:

LUT (6.72%)：主要負擔組合邏輯運算。由於 IM 與 DM 採用暫存器陣列實作，導致大量的讀取邏輯（MUX）佔用此類資源。

FF (1.14%)：主要儲存序向邏輯狀態。包含 PC、Register File 中的 32 個暫存器以及記憶體模組中的儲存單元。

IO (41.25%)：為佔用率最高之項目。主因為頂層模組引出了多組 32-bit 匯流排（Output_Addr, Input_Addr, ext_data 等）。

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 9.672 ns	Worst Hold Slack (WHS): 0.942 ns	Worst Pulse Width Slack (WPWS): 19.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 9280	Total Number of Endpoints: 9280	Total Number of Endpoints: 3073

All user specified timing constraints are met.

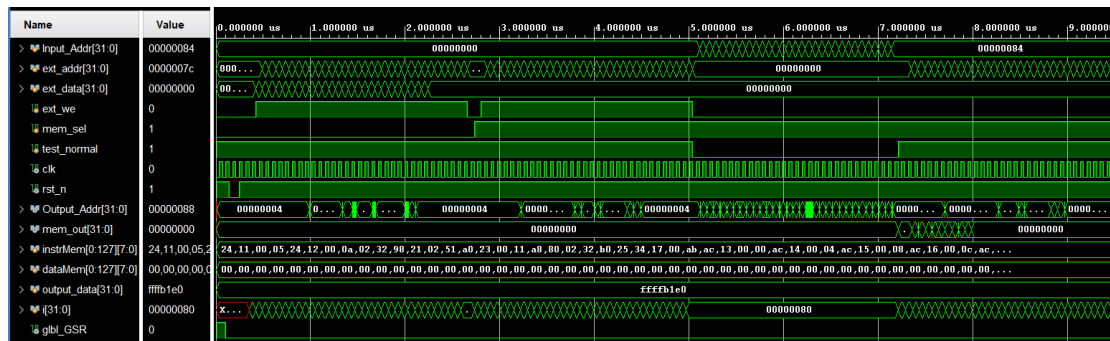
Timing Report :

WNS：9.672 ns，Setup Time 滿足要求

WHS：0.942 ns。Hold Time 滿足要求，確保資料在時脈邊緣觸發時能穩定被取樣

WPWS：19.500 ns。反映時脈訊號本身的穩定性與脈衝完整性符合規範

4. Post-Synthesis Timing Simulation (part 3)



載入階段 (0-5.5 μ s)：test_normal=1，透過 ext_we 脈衝將指令 (IM.dat) 與資料 (DM.dat) 成功載入記憶體。

執行階段 (5.5 μ s 後)：test_normal=0，PC (Input_Addr) 由 00000000 穩定遞增至 00000084；Output_Addr 預先計算下一位址，證實分支與跳轉邏輯正常。

2. Vivado Tcl 控制台分析

- 模擬結束點：於 9520 ns 觸發 \$finish
- 模擬效能：執行時間約 17 分 19 秒，反映門級延遲計算對運算資源的負擔

Simulation Completed. By IEESLab

```

/\_/\  /\_/\  /\_/\  Congratulations !
( o.o ) ( -.- ) ( ^_^ ) Please Check <<DM.out>>
> ^ <  > ^ <  > ^ <  Wishing you continued success and happiness. By IEESLab

```

```

$finish called at time : 9520 ns : File "C:/Users/USER/Desktop/PA2/TESTBED_v1/tb_SimpleCPU.v" Line 208
run: Time (s): cpu = 00:00:03 ; elapsed = 00:00:06 . Memory (MB): peak = 3185.602 ; gain = 11.898
xsim: Time (s): cpu = 00:00:10 ; elapsed = 00:00:18 . Memory (MB): peak = 3185.602 ; gain = 134.750
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_SimpleCPU_time_synth' loaded.
INFO: [USF-XSim-97] XSim simulation ran for all
launch_simulation: Time (s): cpu = 00:01:36 ; elapsed = 00:17:19 . Memory (MB): peak = 3185.602 ; gain = 1463.051

```

時序收斂：在硬體延遲環境下，位址跳轉與數據存取均無時序違規

功能正確：指令執行流程與 DM_test.out 最終運算結果均符合預期

5. Conclusion

在本次 SimpleCPU 實作過程，深刻體會到硬體設計與純軟體開發的本質差異。最令我印象深刻的莫過於執行 Post-Synthesis Timing Simulation 的時間成本，行為模擬的數秒運算，變成了動輒十分鐘、甚至長達十七分鐘的等待。看著遲遲不動的進度條，常常懷疑 Vivado 是否當機，等待時間都可以拿來寫電磁學作業了，這與以往寫程式即時回饋的經驗截然不同。

此外，有別於以往助教直接提供驗證用的 .dat 檔，這次須自己撰寫機器碼測試 CPU 功能。這個過程讓我被迫更深入地檢視 CPU 運作的細節，並開始思

考如何精準地構造指令序列來驗證特定功能。這種自己餵資料的驗證過程，讓我對自己實作的硬體架構有了前所未有的清晰感，也學會了如何從驗證者的角度去思考系統的完整性與穩定性。

在功能驗證中，Branch 指令無疑是整個模擬過程中最耗時且複雜的環節。它需要動用最龐大的判斷路徑，包含 ALU 的減法運算、位址偏移量的加法器以及決定 PC 跳轉的多工器。模擬器必須逐一計算每一個邏輯門的物理延遲，這正是效能損耗的根源，也讓我理解到硬體資源與運算時間之間的權衡關係。